Clerk — a Dispute Intake Assistant

Project report · Introduction to Generative AI · University of Debrecen · Fall 2026

Muhammad Muneeb Naveed (solo project)

October 2026

1. Problem and motivation

When a cardholder disputes a card payment, the first step is *intake*: the issuing bank has to turn a complaint such as “someone charged me 180 euros at a shop I’ve never heard of” into a case it can act on. That needs a fixed set of facts that depends on the kind of dispute: the merchant, amount, currency and date; whether the card is still in the cardholder’s possession; whether anyone else was allowed to use it; whether the merchant was contacted. Cardholders rarely volunteer all of these, often state them vaguely, and sometimes contradict themselves. A human agent spends the call asking for what is missing and noticing what does not add up.

Clerk automates that conversation. It reads the cardholder’s own words, fills a structured case file, asks one plain-language question at a time for whatever is still missing, and flags inconsistencies for a human reviewer. It does not decide the dispute, which stays with a person. Large language models are good at the part that is hard to code (understanding free text in any phrasing, including Hungarian) and bad at the part that must be trustworthy (never guessing a fact, applying fixed rules the same way every time). The design follows from that split.

The domain comes from my work and my thesis, an issuer-side dispute-resolution engine. This project borrows only the domain idea: separate repository, schema and code, no shared interfaces (`docs/decisions.md`, D1–D3).

2. Approach in one sentence: the model extracts, the code decides

Each turn makes **one** call to Claude Opus 5.5. The model returns a schema-constrained object: the case file as it now stands, any contradictions it noticed (each quoting its evidence), and one proposed next question. Every field is optional, and anything the cardholder has not said must stay `null`. Everything that must be reliable is plain Python with unit tests: which fields each dispute type requires, whether the case is complete, which dispute regime applies, the hard inconsistency rules, which question is actually asked, and an 8-turn cap (D4).

Alternatives considered: letting the model run the whole conversation and decide completeness (unverifiable and not reproducible), and an agent with tools such as `set_field` and `mark_complete` (every decision would still be the model’s, and harder to evaluate).

3. Architecture

```
browser ─► Streamlit (app.py) ─► IntakeSession ──────────► extract_turn ─► Claude Opus 5.5
                                  | merge known fields        one structured call per turn
                                  | missing fields → complete?
                                  | question: required + still missing, else fallback
                                  ├─ select_regime  (regime.py)
                                  └─ hard_flags     (rules.py)
```

Module	Role
`intake/schema.py`	Pydantic models. Every leaf optional. Required fields per dispute type.
`intake/prompt.py`	The frozen system prompt and the wrappers that delimit cardholder text.
`intake/extract.py`	The only module that calls the API: structured output, effort, caching, typed errors.
`intake/session.py`	State and decisions: merge, completeness, question choice, flags, sanitising.
`intake/regime.py`, `rules.py`	Pure functions: dispute regime (EU card, US card, SEPA direct debit) and four hard rules.
`app.py`	Streamlit UI: chat, live case file, flags, settings sidebar, JSON export.
`eval/`	22-case golden set, scripted cardholder, deterministic scorer, LLMail-Inject replay.

A turn. The cardholder's message is wrapped in `<cardholder_message>` tags and appended to the history; the first message also carries the filing date (§4). The model re-reads the whole conversation and returns an `ExtractionResult`. The session merges it into the case file (a value once stated is never lost when a later turn leaves it out), overwrites the narrative with the cardholder's verbatim words, strips links and e-mail addresses from model-written text, computes the missing required fields, and either finishes or asks the next question. If the model's question targets anything other than a required, still-missing field, a pre-written question for the first missing field is used instead. A failed API call changes nothing, so a retry is clean.

Demo. The left pane is the chat; the right pane shows the dispute type, the regime, a checklist of required fields filling in, and the reviewer flags, all updated every turn. A "model's last proposal" expander shows the raw extraction next to what the code decided.

4. Prompt design

The course's prompt-engineering material names zero-shot, one-shot, few-shot, persona and chain of thought as the core techniques, together with the **3C principle** (Clarity, Context, Constraints) and iterative refinement. Clerk uses:

- **Persona.** "You are the intake assistant on a card issuer's dispute desk", with explicit limits: it does not decide the dispute, give advice or promise outcomes.
- **Few-shot.** Two worked examples. The first shows a messy complaint where most fields must stay `null` ("last week" is not one day, so the date stays empty); the second shows an explicit "I haven't contacted them" recorded as `false`, not `null`. Example values never appear in the eval cases, so the examples cannot leak answers.
- **Clarity.** Definitions of the six dispute types, and the central **null rule**: *"Fill a field only when the cardholder has stated it. If it is not stated, leave it null. Never infer amounts, dates, or yes/no answers."* A nullable schema gives the model a legal way to say "not stated" for every field.
- **Context.** The role, the dispute types, and the **filing date**, which lets "yesterday" become a real date. The date travels in a `<filing_date>` tag in the first user message, not in the system prompt: a date in the system prompt would change it every day and defeat prompt caching (D12). Vague references such as "a while ago" stay `null`.

- **Constraints.** Exactly one question per turn, plain language, the answer as a dotted field path, and a JSON schema enforced by structured output (constrained decoding, which guarantees shape but not truth; the eval measures truth).
- **Iterative refinement, measured.** Each prompt change below was driven by an observed failure and checked afterwards: the field-path format ("case_file.dispute_type" seen live), the payment-method wording (eval v1), and the free-text rule for quoted messages (LLMail-Inject v1).

Supporting techniques: delimiting untrusted input (spotlighting), grounded contradiction detection (every semantic flag must quote the conflicting statements), and prompt caching of the ≈4,700-token system prompt (the second request reads it from cache: about 5 s instead of 24 s).

Chain of thought. Opus 5.5 always reasons before answering (adaptive thinking); the depth is set with `effort` rather than by writing "think step by step". I also tried an explicit, visible version: a required `reasoning` field placed first in the output schema, since output is generated in field order. Opus 5.5 refused the request (`stop_reason: "refusal"`) on the plain smoke-test complaint, most likely because its safety classifiers treat requests to write out its reasoning as reasoning extraction. Without the field the same request succeeds, so it was reverted and recorded as a negative result (decisions record, open questions).

5. Parameters

All settings live in `intake/config.py`, can be overridden by environment variables, and are shown in the sidebar, locked once a conversation starts because prompt caches are per model.

Parameter	Where	Effect
`effort` (low…max)	Opus 5.5	How much the model thinks before answering. Default `medium`.
`max_tokens`	both	Upper bound on the response; the eval runs at 16,000 so answers are never cut.
`temperature`	Sonnet 4.6	Scales the next-token distribution: 0 is near-greedy, 1 is as trained.
`top_p`	Sonnet 4.6	Samples only from the smallest set of tokens whose probability reaches *p*.
turn cap, timeout, retries	session / client	8 turns; 90 s timeout; 2 automatic retries.

Opus 5.5 does not accept sampling parameters, so the sidebar offers a second path on Claude Sonnet 4.6, same prompt and schema, with temperature and top_p (D11). A finding while building it: the current API version (`2026-10-21`, sent by the SDK) rejects `temperature` on *every* model, including Sonnet 4.6. The older version `2023-06-01` still accepts it, so the sampling path pins that version for its own requests. Low temperature is the right setting for extraction (the same complaint should give the same case file); temperature is shown to demonstrate the parameter, not because extraction benefits from it.

6. Error handling

Situation	Handling
Rate limit, timeout, connection error	Retried twice by the SDK with backoff, then a readable message and a “Retry last message” button.
Empty or blank input	Refused by the chat box, and again by `IntakeSession.step` before any API call.
Invalid API key, other API errors	A message naming the cause; no traceback.
Refusal, output cut off by `max_tokens`, output failing validation	Separate messages; out-of-range values (e.g. a negative amount) fail Pydantic validation.
Any failure mid-conversation	The session is left unchanged, so retrying never duplicates a message.

All of these are unit-tested with a fake client; a forced timeout (`TIMEOUT_S=0.5`) was also checked in the browser.

7. Evaluation

Set-up. 22 hand-written cases: at least three per dispute type, one per hard rule, three “invention traps” in which a date is never given, two SEPA direct debits, two US-issued cards, one in Hungarian, one that starts with the wrong dispute type and corrects itself, and two prompt injections. A scripted cardholder replays each case against the real session: it answers only the field Clerk asks about and says “I don’t know” otherwise, so a value appearing for an unanswered field was invented. The labels list only stated facts; every other field must stay empty. A unit test recomputes each label’s regime and hard flags with the real `regime.py` and `rules.py`, so the labels cannot drift from the code.

Scoring is exact comparison, not an LLM judge: the outputs are structured, so a judge would add cost and a second source of error (D8). Metrics: field accuracy, *invented* values (the hallucination measure), missed and wrong values, flag precision/recall (hard rules and injection), regime, completion, turns.

Metric	v1	v2
Field accuracy	98.6%	**100%**
Invented values	0	**0**
Missed values	5	**0**
Flag precision / recall	100% / 100%	100% / 100%
Regime correct	20/22	**22/22**
Required fields collected (where possible)	18/22	**22/22**
Mean turns	5.8	5.4

Error analysis of v1. Every miss was read in its transcript. (1) In the unauthorized cases the model asked about *optional* fields (e.g. “have you disputed this merchant before?”), which used up the 8-turn cap before required fields were asked: four cases incomplete, two regimes undetermined. I had allowed such questions when building the session; the evidence reversed that decision, and the session now accepts the model’s question only for a required, missing field. (2) “A charge on my card” was not read as a card payment; one prompt line fixed it.

Disclosure. v1 as run counted one invented value: “*my* electricity company Voltaria” was recorded as recognising the merchant. That follows from the prompt’s own rule (“a statement that plainly means it”), so it was a labelling problem. The field is unscored in the three cases worded that way, and v1 is reported re-scored with the final labels (`eval/results-v1-rescored.md`; the raw run is kept in `results-v1.md`).

Limits of the evaluation. One run per version, and the model is not deterministic. The cases are self-written, by the author of the prompt, and are not real complaints. The result shows that the design works on these cases, not that it generalises.

8. Security: prompt injection

Cardholder text is untrusted input placed inside a prompt, and complaints often quote merchant e-mails, so injection is a real threat. Defences, cheapest first: instructions only in the system block; every cardholder message wrapped in `<cardholder_message>` tags and declared as data (a message cannot close its own tag); schema-constrained output with no free-text channel; all decisions in code, so an injection cannot mark a case complete or change the regime; the model's output validated as untrusted too (field whitelist, enum, date and range checks, question length capped and shown as plain text, links and e-mail addresses removed from free-text fields); the narrative set by code; an `INSTRUCTION_IN_INPUT` flag so attempts are visible; no tools, so nothing can be triggered.

Test with real attacks. From Microsoft's public LLMail-Inject challenge (arXiv 2506.09956) I sampled 3,000 submissions and kept the 35 unique attacks that had *succeeded* against the challenge's defended e-mail assistant, making it send an e-mail to the attacker undetected. Each was replayed against Clerk twice: as the cardholder's whole message, and pasted into a genuine complaint as "the merchant's reply".

Run	Framing	Flagged	Fields invented	Genuine facts intact	Attacker address in model output / final case file
v1	direct	35/35	0/35	n/a	0/35 / 0/35
v1	indirect	35/35	0/35	35/35	2/35 / 2/35
v2	indirect	35/35	0/35	35/35	**0/35 / 0/35**

The attackers' goal is impossible by construction, as Clerk cannot send anything. In v1 the model twice copied the attacker's address into its summary of the merchant's reply. v2 added two layers: the prompt defines that field as the merchant's *position* in the model's own words and forbids copying anything from quoted messages, which alone brought leaks to zero; and the session now removes e-mail addresses (including spaced-out ones) and links from all model-written free text, which guarantees the result even when the model does not comply. The two eval injections were also flagged with no fields filled from the instruction.

9. Limitations and ethics

- **The model only proposes.** Clerk collects facts and flags doubts; a human decides. A wrong extraction costs a reviewer's time, not a cardholder's money.
- **Single provider.** Everything depends on one model family; the safety refusal described in §4 shows that model behaviour can change what a design is allowed to do.
- **No personal-data handling.** Sessions live in memory and vanish on restart; nothing is logged. A real issuer would need authentication, an audit log, retention rules and GDPR-compliant storage, and would connect to its transaction records instead of trusting the cardholder's figures.
- **Regimes are simplified labels**, not legal advice.
- **Cost and latency.** About 5–25 s per turn; the 22-case eval costs a few dollars.

10. Future work

A reviewer screen that shows the transcript, flags and evidence side by side; prefilling from the bank's transaction record so the cardholder only confirms; evaluation on real, anonymised complaints and on more languages; repeated eval runs to report spread.

11. References

- Anthropic. *Claude API documentation*: structured outputs, prompt caching, adaptive thinking and effort, refusals and fallbacks. platform.claude.com/docs
- Abdelnabi, S., Fay, A., Salem, A., et al. (2025). *LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge.* arXiv:2506.09956. Dataset: huggingface.co/datasets/microsoft/llmail-inject-challenge (MIT)
- University of Debrecen, *Introduction to Generative AI* (Fall 2026): project grading criteria; "Introduction to Generative Artificial Intelligence: The Basics of Prompt Engineering".
- Directive (EU) 2015/2366 (PSD2), Art. 73, on refunds for unauthorised payment transactions.
- Source code, eval data and results: the project repository (`README.md`, `docs/decisions.md`, `eval/`).